

3차 과제 ChatGPT 대화 기록

공부 캐릭터 성장 프로그램 - 전체 대화 정리본

아래 내용은 현재 대화에서 주고받은 질문과 답변을 과제 제출용으로 읽기 쉽게 정리한 PDF입니다.

사용자

저번에 이어서 3차과제야 파일 올려줄게 3차 과제 조건을 봤는데 while True, break, 함수 3개 이상, 매개변수, return, global을 꼭 넣어야 한대. 이게 각각 무슨 뜻인지 쉽게 설명해줘. 글고 내 2차 과제 코드는 공부 캐릭터 성장 프로그램이었는데, 이걸 3차 과제 조건에 맞게 방향이 좋을까?

ChatGPT

좋아. 3차 과제 조건은 2차 과제 코드를 "메뉴형 프로그램"으로 발전시키면 자연스럽게 다 넣을 수 있어.
네 2차 과제가 공부 캐릭터 성장 프로그램이었으니까, 3차에서는 사용자가 계속 메뉴를 고르고, 공부 기록을 입력하면 캐릭터 경험치/레벨/달성률 만들면 딱 좋아.

1. `while True` 뜻

`while True`는 무한 반복문이야.

```
```python
while True:
 print("계속 반복됩니다")
```
```

`True`는 항상 참이니까 프로그램이 계속 돌아가.
그래서 보통 메뉴 프로그램에서 많이 써.

예를 들면:

```
```python
while True:
 print("1. 공부하기")
 print("2. 상태보기")
 print("3. 종료하기")
```
```

이렇게 하면 사용자가 종료를 선택하기 전까지 메뉴가 계속 나와.

2. `break` 뜻

`break`는 반복문을 멈추고 빠져나가는 명령어야.

```
```python
while True:
 menu = input("메뉴 선택: ")

 if menu == "3":
 print("프로그램을 종료합니다.")
 break
```
```

여기서 사용자가 ``3``을 입력하면 `break`가 실행되고, `while True` 반복이 끝나.

즉,

```
```python
while True:
```
```

는 계속 반복시키는 역할이고,

```
```python
break
```
```

는 그 반복을 끝내는 역할이야.

3차 과제에서는 메뉴에 종료하기를 넣으면 자연스럽게 사용할 수 있어.

3. 함수 3개 이상 뜻

함수는 자주 쓰는 코드를 이름 붙여서 따로 정리한 것이야.

예를 들어 캐릭터 상태를 보여주는 코드를 매번 길게 쓰기 귀찮으니까 함수로 만들 수 있어.

```
```python
def show_status():
 print("캐릭터 상태를 보여줍니다")
```
```

그리고 필요할 때 이렇게 불러.

```
```python
show_status()
```
```

3차 과제 조건이 함수 3개 이상이면, 예를 들어 이렇게 만들 수 있어.

```
```python
def show_menu():
 # 메뉴 출력

def study():
 # 공부 시간 입력받고 경험치 증가

def show_status():
 # 캐릭터 상태 출력
```
```

네 프로그램에서는 함수 3개를 억지로 넣는 게 아니라, 기능별로 나누면 자연스러워.

4. 매개변수 뜻

매개변수는 함수에게 전달해주는 값이야.

예를 들어 이름을 받아서 인사하는 함수를 만든다고 해보자.

```
```python
def hello(name):
 print(name, "님 안녕하세요!")
```
```

여기서 `name`이 매개변수야.

사용할 때는 이렇게 해.

```
```python
hello("민석")
```
```

그러면 출력은:

```
```python
민석 님 안녕하세요!
```
```

네 과제에서는 예를 들어 공부 시간을 함수에 전달할 수 있어.

```
```python
def get_exp(study_time):
 exp = study_time * 10
 return exp
```
```

여기서 `study\_time`이 매개변수야.

즉, 함수 안에서 사용할 값을 바깥에서 넣어주는 역할이야.

5. `return` 뜻

`return`은 함수에서 계산한 결과를 다시 돌려주는 것이야.

```
```python
def get_exp(study_time):
 exp = study_time * 10
 return exp
```
```

이 함수는 공부 시간을 받아서 경험치를 계산하고, 그 결과를 돌려줘.

사용할 때는 이렇게 해.

```
```python
plus_exp = get_exp(2)
print(plus_exp)
```
```

출력:

```
```python
20
```
```

즉, `return`은 함수가 만든 값을 바깥에서 다시 사용할 수 있게 해주는 역할이야.

네 프로그램에서는 이런 식으로 쓸 수 있어.

```
```python
def calculate_rate(done, goal):
 rate = done / goal * 100
 return rate
```
```

이렇게 하면 달성률을 계산해서 다시 받을 수 있어.

6. `global` 뜻

`global`은 함수 안에서 바깥에 있는 변수를 수정하고 싶을 때 쓰는 것이야.

예를 들어 캐릭터 경험치가 함수 밖에 있다고 해보자.

```
```python
exp = 0

def study():
 global exp
 exp += 10
```
```

여기서 `global exp`를 쓰면 함수 안에서도 바깥에 있는 `exp`를 바꿀 수 있어.

만약 `global`을 안 쓰고 함수 안에서 `exp += 10`을 하면 오류가 날 수 있어.

왜냐하면 파이썬은 함수 안의 `exp`와 함수 밖의 `exp`를 다른 변수로 생각할 수 있기 때문이야.

네 과제에서는 이런 변수에 `global`을 쓰면 좋아.

```
```python
level
exp
total_study_time
```
```

예를 들어:

```
```python
level = 1
exp = 0

def study():
 global exp, level
 exp += 20

 if exp >= 100:
 level += 1
 exp = 0
```
```

이렇게 하면 공부할 때마다 경험치가 오르고, 경험치가 100 이상이면 레벨업할 수 있어.

2차 과제를 3차 과제로 발전시키는 방향

네 2차 과제가 공부 캐릭터 성장 프로그램이었다면, 3차에서는 단순 입력/출력에서 끝내지 말고 사용자가 메뉴를 선택하면서 계속 사용할 수 있는 좋아.

예를 들면 이런 방향이 자연스러워.

3차 과제 프로그램 방향

프로그램 이름은 예를 들어:

공부 캐릭터 성장 관리 프로그램

또는

나의 공부 성장 게임

정도로 하면 돼.

기능은 이렇게 구성하면 좋아.

```
```python
1. 공부하기
2. 캐릭터 상태 보기
3. 오늘의 목표 확인하기
4. 보상 메시지 보기
5. 종료하기
```
```

사용자가 `1`을 선택하면 공부 시간을 입력하고, 그 시간에 따라 경험치가 올라가.

사용자가 `2`를 선택하면 캐릭터 이름, 레벨, 경험치, 총 공부 시간, 목표 달성률을 보여줘.

사용자가 `3`을 선택하면 오늘 목표와 현재 공부량을 비교해서 얼마나 달성했는지 알려줘.

사용자가 `4`를 선택하면 달성률에 따라 보상 메시지를 출력해.

사용자가 `5`를 선택하면 `break`로 프로그램이 종료돼.

```

---

## 조건을 자연스럽게 넣는 방법

### `while True`

메뉴가 계속 반복되도록 사용.

```python
while True:
 show_menu()
```

---

### `break`

사용자가 종료 메뉴를 선택했을 때 사용.

```python
if menu == "5":
 break
```

---

### 함수 3개 이상

기능별로 함수 만들기.

```python
def show_menu():
 메뉴 출력

def study():
 공부 시간 입력 및 경험치 증가

def show_status():
 캐릭터 상태 출력

def calculate_rate(done, goal):
 달성률 계산

def reward_message(rate):
 보상 메시지 출력
```

이렇게 하면 함수가 5개라서 조건을 충분히 만족해.

---

### 매개변수

달성률 계산 함수나 보상 메시지 함수에서 사용.

```python
def calculate_rate(done, goal):
 return done / goal * 100
```

여기서 `done`, `goal`이 매개변수야.

---

### `return`

달성률이나 경험치 계산 결과를 돌려줄 때 사용.

```python
def calculate_exp(study_time):

```

```

 return study_time * 10
...

```

---

### `global`

공부 시간, 경험치, 레벨처럼 프로그램 전체에서 계속 바뀌는 값에 사용.

```

```python
def study():
    global total_study_time, exp, level
...

```

추천 발전 방식

2차 과제에서는 아마 사용자의 이름, 목표, 공부 시간, 캐릭터 상태 등을 입력받고 출력하는 정도였다면, 3차 과제에서는 이것을 게임처럼 반복 바꾸면 돼.

예를 들면 이런 식이야.

- > 2차 과제에서는 사용자의 공부 목표와 캐릭터 정보를 입력받아 간단히 출력하는 프로그램을 만들었다.
- > 3차 과제에서는 이를 발전시켜 사용자가 메뉴를 직접 선택하며 공부 시간을 기록하고, 공부 시간에 따라 경험치와 레벨이 증가하는 성장형 프로그램으로 만들었다.
- > `while True`를 사용하여 프로그램이 계속 실행되도록 하고, 사용자가 종료를 선택하면 `break`로 반복을 끝낸다.
- > 또한 메뉴 출력, 공부 기록, 상태 확인, 달성률 계산, 보상 메시지 출력 기능을 함수로 나누어 코드의 구조를 더 깔끔하게 만든다.
- > 공부 시간과 목표 시간을 매개변수로 전달하여 달성률을 계산하고, `return`을 사용해 계산 결과를 다시 받아 활용한다.
- > 경험치, 레벨, 총 공부 시간처럼 프로그램 전체에서 계속 바뀌는 값은 `global`을 사용해 함수 안에서 수정할 수 있도록 한다.

이렇게 쓰면 저번 과제에서 발전했다는 느낌도 나고, 3차 조건도 전부 자연스럽게 들어가.

제일 추천하는 3차 과제 구조

```

```python
기본 정보 입력
user_name = input("사용자 이름을 입력하세요: ")
goal_time = int(input("오늘 목표 공부 시간을 입력하세요: "))

```

```

level = 1
exp = 0
total_study_time = 0

```

# 함수들

```

def show_menu():
 메뉴 출력

```

```

def calculate_exp(study_time):
 경험치 계산 후 return

```

```

def calculate_rate(total, goal):
 달성률 계산 후 return

```

```

def study():
 global 사용해서 공부 시간, 경험치, 레벨 변경

```

```

def show_status():
 현재 상태 출력

```

```

def reward_message(rate):
 달성률에 따른 메시지 출력

```

# 메인 반복문

```

while True:
 show_menu()
 menu = input("메뉴 선택: ")

 if menu == "1":

```

```

 study()
 elif menu == "2":
 show_status()
 elif menu == "3":
 달성률 확인
 elif menu == "4":
 보상 메시지 확인
 elif menu == "5":
 break
 ...

```

이 방향으로 만들면 과제 조건이 전부 들어가고, 2차 과제보다 확실히 발전한 프로그램처럼 보여.

## 사용자

메뉴형 프로그램이라는 게 정확히 무슨 뜻이야? 내 공부 캐릭터 프로그램에 적용하면 메뉴를 어떻게 구성하면 좋을까? 그리고 전역변수랑 지역변 프로그램에서는 어떤 변수를 전역변수로 두고, 어디에서 global을 쓰면 좋을까?

## ChatGPT

메뉴형 프로그램은 쉽게 말해서 사용자가 번호를 골라서 원하는 기능을 실행하는 프로그램이야.

예를 들어 음식점 키오스크처럼,

```

```python
1. 공부하기
2. 상태 보기
3. 목표 달성률 보기
4. 보상 메시지 보기
5. 종료하기
```

```

이런 식으로 선택지가 계속 나오고, 사용자가 `1`, `2`, `3` 같은 번호를 입력하면 그 기능이 실행되는 구조야.

---

# 1. 메뉴형 프로그램이 왜 3차 과제에 잘 맞나면

3차 과제 조건에 있는 것들이 메뉴형 프로그램에서 자연스럽게 들어가.

```

```python
while True
```

```

메뉴를 계속 보여줄 때 사용해.

```

```python
break
```

```

사용자가 종료를 선택했을 때 프로그램을 끝낼 때 사용해.

```

```python
함수 3개 이상
```

```

메뉴 출력, 공부하기, 상태 보기 같은 기능을 함수로 나누면 돼.

```

```python
매개변수, return
```

```

공부 시간으로 경험치를 계산하거나, 목표 시간과 공부 시간을 이용해 달성률을 계산할 때 쓰면 돼.

```

```python
global
```

```

공부 시간이 늘어나거나, 경험치와 레벨이 바뀔 때 쓰면 돼.

---

## # 2. 네 공부 캐릭터 프로그램에 적용한 메뉴 예시

네 프로그램이 공부 캐릭터 성장 프로그램이니까 메뉴는 이렇게 구성하면 좋아.

```
```python
===== 공부 캐릭터 성장 프로그램 =====
1. 공부하기
2. 캐릭터 상태 보기
3. 목표 달성률 보기
4. 보상 메시지 보기
5. 종료하기
```
```

각 메뉴의 역할은 이렇게 생각하면 돼.

---

### ## 1번. 공부하기

사용자가 공부한 시간을 입력하는 기능이야.

예를 들어:

```
```python
공부한 시간을 입력하세요: 2
```
```

그러면 총 공부 시간이 증가하고, 경험치도 증가해.

예시:

```
```python
2시간 공부했습니다!
경험치가 20 증가했습니다!
```
```

여기서 바뀌는 값은:

```
```python
total_study_time
exp
level
```
```

이런 것들이야.

---

### ## 2번. 캐릭터 상태 보기

현재 캐릭터 정보를 보여주는 기능이야.

예를 들어:

```
```python
이름: 민석
레벨: 2
경험치: 40
총 공부 시간: 5시간
오늘 목표 시간: 8시간
```
```

공부 게임 느낌을 내려면 이 메뉴가 꼭 있는 게 좋아.

---



## 3번. 목표 달성률 보기

현재까지 공부한 시간이 목표의 몇 퍼센트인지 보여주는 기능이야.

예를 들어 목표가 10시간이고 지금까지 4시간 공부했다면:

```
```python
현재 달성률은 40.0%입니다.
```
```

이 기능은 `return`을 쓰기 좋아.

```
```python
def calculate_rate(total, goal):
    rate = total / goal * 100
    return rate
```
```

---

## 4번. 보상 메시지 보기

달성률에 따라 다른 메시지를 출력하는 기능이야.

예를 들어:

```
```python
달성률 30% 미만: 아직 시작 단계예요!
달성률 70% 미만: 잘하고 있어요!
달성률 100% 이상: 목표 달성! 보상을 획득했습니다!
```
```

이 기능을 넣으면 단순한 공부 기록 프로그램보다 캐릭터 성장 게임 느낌이 더 나.

---

## 5번. 종료하기

프로그램을 끝내는 기능이야.

여기서 `break`를 쓰면 돼.

```
```python
elif menu == "5":
    print("프로그램을 종료합니다.")
    break
```
```

---

# 3. 전역변수와 지역변수 쉽게 설명

## 전역변수

전역변수는 프로그램 전체에서 같이 사용하는 변수야.

함수 밖에 만들어.

```
```python
level = 1
exp = 0
total_study_time = 0
```
```

이런 변수들은 프로그램이 실행되는 동안 계속 유지되어야 해.

예를 들어 공부하기 메뉴에서 공부 시간이 늘어나고, 상태 보기 메뉴에서 그 늘어난 값을 보여줘야 하잖아?

그러려면 여러 함수가 같이 쓸 수 있게 함수 밖에 두는 게 좋아.

---

## 지역변수

지역변수는 함수 안에서만 잠깐 쓰는 변수야.

```
```python
def calculate_exp(study_time):
    plus_exp = study_time * 10
    return plus_exp
```
```

여기서 `plus\_exp`는 이 함수 안에서만 쓰는 변수야.

함수가 끝나면 바깥에서는 직접 사용할 수 없어.

쉽게 말하면:

```
```python
지역변수 = 프로그램 전체 공용 노트
지역변수 = 함수 안에서만 쓰는 메모지
```
```

라고 생각하면 돼.

---

# 4. 네 프로그램에서 전역변수로 두면 좋은 것

네 공부 캐릭터 프로그램에서는 이런 것들을 전역변수로 두면 좋아.

```
```python
user_name
goal_time
level
exp
total_study_time
```
```

각각의 의미는 이거야.

```
```python
user_name
```
```

사용자 이름. 여러 화면에서 보여줄 수 있음.

```
```python
goal_time
```
```

오늘 목표 공부 시간. 달성률 계산에 계속 필요함.

```
```python
level
```
```

캐릭터 레벨. 공부할 때마다 바뀔 수 있음.

```
```python
exp
```
```

경험치. 공부할 때마다 증가함.

```
```python
total_study_time
```
```

총 공부 시간. 공부하기 메뉴를 누를 때마다 증가함.

---

# 5. 어디에서 `global`을 쓰면 좋을까?

중요한 기준은 이거야.

함수 안에서 전역변수 값을 바꾸면 `global`이 필요해.

보기만 하면 보통 `global`이 없어도 돼.  
수정하면 `global`이 필요해.

---

## `global`이 필요한 함수

공부하기 함수에서는 전역변수 값을 바꿔야 해.

```
```python
def study():
    global total_study_time, exp, level
```
```

왜냐하면 공부하기를 하면

```
```python
total_study_time += study_time
exp += plus_exp
level += 1
```
```

이렇게 값이 바뀌기 때문이야.

그래서 `study()` 함수에는 `global`을 쓰는 게 좋아.

---

## `global`이 없어도 되는 함수

상태 보기 함수는 값을 바꾸는 게 아니라 보기만 해.

```
```python
def show_status():
    print(level)
    print(exp)
    print(total_study_time)
```
```

이런 경우에는 보통 `global`이 없어도 돼.

달성률 계산 함수도 값을 직접 바꾸는 게 아니라 계산만 해.

```
```python
def calculate_rate(total, goal):
    return total / goal * 100
```
```

여기서는 `global`을 쓰지 않는 게 더 좋아.

왜냐하면 `total`, `goal`을 매개변수로 받아서 계산하고, 결과만 `return`으로 돌려주기 때문이야.

---

# 6. 네 프로그램 기준으로 정리하면

이렇게 나누면 돼.

```
```python
# 전역변수
user_name = input("사용자 이름을 입력하세요: ")
goal_time = int(input("오늘 목표 공부 시간을 입력하세요: "))
```

```

level = 1
exp = 0
total_study_time = 0
...

```

그리고 함수는 이렇게 나뉘.

```

```python
def show_menu():
 # 메뉴 출력
...

```

메뉴만 출력하니까 `global` 필요 없음.

```

```python
def calculate_exp(study_time):
    # 공부 시간으로 경험치 계산
    return study_time * 10
...

```

계산만 하니까 `global` 필요 없음.

```

```python
def calculate_rate(total, goal):
 # 달성률 계산
 return total / goal * 100
...

```

계산만 하니까 `global` 필요 없음.

```

```python
def study():
    global total_study_time, exp, level
    # 공부 시간, 경험치, 레벨을 바꿈
...

```

전역변수 값을 바꾸니까 `global` 필요함.

```

```python
def show_status():
 # 캐릭터 상태 출력
...

```

값을 보기만 하니까 `global` 필요 없음.

---

# 7. 전체 구조를 간단히 보면

```

```python
user_name = input("사용자 이름: ")
goal_time = int(input("목표 공부 시간: "))

```

```

level = 1
exp = 0
total_study_time = 0

```

```

def show_menu():
    print("1. 공부하기")
    print("2. 캐릭터 상태 보기")
    print("3. 목표 달성률 보기")
    print("4. 보상 메시지 보기")
    print("5. 종료하기")

```

```

def calculate_exp(study_time):
    return study_time * 10

```

```

def calculate_rate(total, goal):
    return total / goal * 100

```

```

def study():
    global total_study_time, exp, level

    study_time = int(input("공부한 시간 입력: "))
    total_study_time += study_time

    plus_exp = calculate_exp(study_time)
    exp += plus_exp

    if exp >= 100:
        level += 1
        exp = exp - 100
        print("레벨업!")

    print(study_time, "시간 공부했습니다.")

def show_status():
    print("이름:", user_name)
    print("레벨:", level)
    print("경험치:", exp)
    print("총 공부 시간:", total_study_time)
    print("목표 공부 시간:", goal_time)

while True:
    show_menu()
    menu = input("메뉴 선택: ")

    if menu == "1":
        study()
    elif menu == "2":
        show_status()
    elif menu == "3":
        rate = calculate_rate(total_study_time, goal_time)
        print("현재 달성률:", rate, "%")
    elif menu == "4":
        rate = calculate_rate(total_study_time, goal_time)

        if rate >= 100:
            print("목표 달성! 캐릭터가 크게 성장했습니다!")
        elif rate >= 50:
            print("절반 이상 달성했어요! 조금만 더 힘내세요!")
        else:
            print("아직 시작 단계예요. 공부를 더 해봅시다!")

    elif menu == "5":
        print("프로그램을 종료합니다.")
        break
    else:
        print("잘못된 메뉴입니다.")
...

```

결론적으로 네 프로그램에서는 공부하기 기능에서만 `global`을 쓰는 구조가 제일 자연스러워.

`total\_study\_time`, `exp`, `level`은 공부할 때 값이 바뀌니까 전역변수로 두고, `study()` 함수 안에서 `global`로 수정하면 돼.

반대로 `calculate\_exp()`나 `calculate\_rate()`는 계산만 하는 함수니까 매개변수와 `return`을 쓰는 연습용으로 넣으면 과제 조건에 잘 맞아.

사용자

공부 달성률에 따라 보상 메시지를 넣고 싶은데 가능할까? 게임 아이템 말고 실제로 나한테 아이스크림이나 음료를 선물하라는 느낌이면 좋겠어. 4단계로 만들고 싶은데, 단계 이름을 좀 더 재밌게 추천해줘. 너무 유치하지 않고 공부 성장 프로그램에 어울렸으면 좋겠어.

ChatGPT

가능해. 오히려 공부 캐릭터 성장 프로그램이 "게임 속 보상"이 아니라 현실 보상으로 연결되는 느낌이라서 더 자연스럽고 과제 설명에도 좋아 보

예를 들면 공부 달성률에 따라 이런 식으로 메시지를 줄 수 있어.

```
```python
def reward_message(rate):
 if rate >= 100:
 print("완주 단계!")
 print("오늘 목표를 전부 달성했어요. 스스로에게 아이스크림 하나 선물해도 좋아요!")
 elif rate >= 70:
 print("집중 상승 단계!")
 print("목표에 거의 다 왔어요. 좋아하는 음료 한 잔으로 잠깐 쉬어가도 좋아요!")
 elif rate >= 40:
 print("성장 시작 단계!")
 print("공부 흐름이 만들어지고 있어요. 간단한 간식으로 에너지를 채워보세요!")
 else:
 print("출발 준비 단계!")
 print("아직 시작 단계예요. 조금만 더 공부하고 보상을 받아봅시다!")
```
```

여기서 `rate`는 달성률이야.

예를 들어 목표가 5시간이고 지금까지 4시간 공부했다면 달성률은 80%라서 집중 상승 단계 메시지가 나와.

보상 단계 이름 추천

너무 유치하지 않게, 공부 성장 프로그램에 어울리게 만들면 이렇게 추천해.

```
달성률	단계 이름	보상 느낌
0% 이상	출발 준비 단계	아직 시작, 보상은 조금 더 공부한 뒤
40% 이상	성장 시작 단계	간단한 간식 가능
70% 이상	집중 상승 단계	음료 한 잔 보상
100% 이상	목표 완주 단계	아이스크림 같은 확실한 보상
```

개인적으로는 이 조합이 제일 자연스러워.

```
```python
출발 준비 단계
성장 시작 단계
집중 상승 단계
목표 완주 단계
```
```

공부 캐릭터가 성장하는 느낌도 있고, 너무 초등학생 게임처럼 보이지도 않아.

조금 더 세련된 버전

단계 이름을 더 깔끔하게 하고 싶으면 이 버전도 좋아.

```
달성률	단계 이름
0% 이상	워밍업 단계
40% 이상	루틴 형성 단계
70% 이상	몰입 단계
100% 이상	목표 달성 단계
```

이건 좀 더 현실적인 공부 관리 프로그램 느낌이야.

네 과제에 넣기 좋은 최종 추천

나는 이걸 추천해.

```
```python
def reward_message(rate):
 if rate >= 100:
 print("목표 완주 단계")
```
```

```

        print("오늘 목표를 달성했습니다!")
        print("보상: 스스로에게 아이스크림 하나를 선물하세요.")

    elif rate >= 70:
        print("집중 상승 단계")
        print("목표에 거의 도착했습니다!")
        print("보상: 좋아하는 음료 한 잔으로 잠깐 쉬어가세요.")

    elif rate >= 40:
        print("성장 시작 단계")
        print("공부 흐름이 만들어지고 있습니다.")
        print("보상: 작은 간식으로 에너지를 채워보세요.")

    else:
        print("출발 준비 단계")
        print("아직 목표까지는 시간이 더 필요합니다.")
        print("조금 더 공부하면 보상을 받을 수 있어요.")
...

```

그리고 메뉴에서는 이렇게 연결하면 돼.

```

```python
elif menu == "4":
 rate = calculate_rate(total_study_time, goal_time)
 reward_message(rate)
...

```

이렇게 하면 달성을 계산 함수는 `return`을 사용하고, 보상 메시지 함수는 매개변수 `rate`를 받아서 출력하니까 3차 과제 조건에도 잘 맞아.

## 사용자

그럼 2차과제랑 합쳐서 초안 작성해줘

## ChatGPT

2차 과제의 핵심인 "공부 캐릭터 성장"은 유지하고, 3차 조건이 자연스럽게 보이도록 메뉴형 구조로 초안을 잡아볼게. 코드 초안과 과제 설명에 정리해줄게.

찾아보니 2차 과제에는 사용자 이름, 캐릭터 이름/타입, 목표 시간, 과목별 공부 시간, 레벨 · 경험치 · 능력치, 달성을 분석이 들어가 있었어. 이 버리지 않고 "메뉴로 다시 실행할 수 있게" 발전시키는 방향으로 만들면 좋아.

좋아. 아래는 2차 과제 공부 캐릭터 성장 프로그램을 3차 과제 조건에 맞게 합친 초안이야.

2차에서 했던 사용자 이름, 캐릭터 이름, 캐릭터 타입, 목표 시간, 과목별 공부 기록, 레벨/경험치/능력치, 달성을 분석을 유지하고, 3차 조건인 True`, `break`, 함수 3개 이상, 매개변수, `return`, `global`을 자연스럽게 넣은 버전이야.

```

3차 과제 코드 초안

```python
# 공부 캐릭터 성장 프로그램 3차 과제 초안

user_name = input("사용자 이름을 입력하세요: ")
char_name = input("공부 캐릭터 이름을 입력하세요: ")
char_type = input("캐릭터 타입을 입력하세요(꾸준형/스파르타형/자유형): ")
today_goal = int(input("오늘 목표 공부 시간을 입력하세요: "))

level = 1
exp = 0
ability = 10
total_study_time = 0

subjects = []
study_times = []

def show_menu():
    print()

```

```

print("==== 공부 캐릭터 성장 프로그램 ====")
print("1. 공부 기록 입력하기")
print("2. 캐릭터 상태 보기")
print("3. 공부 결과 분석하기")
print("4. 보상 메시지 확인하기")
print("5. 프로그램 종료하기")

def calculate_exp(study_time, char_type):
    if char_type == "스파르타형":
        plus_exp = study_time * 12
    elif char_type == "꾸준형":
        plus_exp = study_time * 10
    else:
        plus_exp = study_time * 8

    return plus_exp

def calculate_total_time(times):
    total = sum(times)
    return total

def calculate_rate(total, goal):
    rate = total / goal * 100
    return rate

def record_study():
    global total_study_time, exp, level, ability

    subject = input("공부한 과목을 입력하세요: ")
    study_time = int(input("공부한 시간을 입력하세요: "))

    subjects.append(subject)
    study_times.append(study_time)

    total_study_time = calculate_total_time(study_times)

    plus_exp = calculate_exp(study_time, char_type)
    exp += plus_exp
    ability += study_time * 2

    print()
    print(subject, "과목을", study_time, "시간 공부했습니다.")
    print("경험치가", plus_exp, "만큼 증가했습니다.")

    if exp >= 100:
        level += 1
        exp -= 100
        ability += 5
        print("레벨업! 현재 레벨이", level, "이 되었습니다.")

def show_status():
    print()
    print("==== 현재 캐릭터 상태 ====")
    print("사용자 이름:", user_name)
    print("캐릭터 이름:", char_name)
    print("캐릭터 타입:", char_type)
    print("레벨:", level)
    print("경험치:", exp)
    print("능력치:", ability)
    print("총 공부 시간:", total_study_time, "시간")
    print("오늘 목표 시간:", today_goal, "시간")

def analyze_result():
    if len(study_times) == 0:

```



```

    print()
    print("아직 공부 기록이 없습니다.")
else:
    total = calculate_total_time(study_times)
    rate = calculate_rate(total, today_goal)

    max_time = max(study_times)
    best_index = study_times.index(max_time)
    best_subject = subjects[best_index]

    sorted_times = sorted(study_times, reverse=True)

    print()
    print("==== 공부 결과 분석 =====")
    print("총 공부 시간:", total, "시간")
    print("목표 달성률:", round(rate, 1), "%")
    print("가장 많이 공부한 과목:", best_subject)
    print("가장 오래 공부한 시간:", max_time, "시간")
    print("공부 시간 내림차순 정렬:", sorted_times)

def reward_message(rate):
    print()
    print("==== 보상 메시지 =====")

    if rate >= 100:
        print("단계: 목표 완주 단계")
        print("오늘 목표를 완전히 달성했습니다!")
        print("보상: 스스로에게 아이스크림 하나를 선물하세요.")

    elif rate >= 70:
        print("단계: 집중 상승 단계")
        print("목표에 거의 도착했습니다.")
        print("보상: 좋아하는 음료 한 잔으로 잠깐 쉬어가세요.")

    elif rate >= 40:
        print("단계: 성장 시작 단계")
        print("공부 흐름이 만들어지고 있습니다.")
        print("보상: 작은 간식으로 에너지를 채워보세요.")

    else:
        print("단계: 출발 준비 단계")
        print("아직 목표까지는 시간이 더 필요합니다.")
        print("조금 더 공부하면 보상을 받을 수 있습니다.")

while True:
    show_menu()
    menu = input("메뉴를 선택하세요: ")

    if menu == "1":
        record_study()

    elif menu == "2":
        show_status()

    elif menu == "3":
        analyze_result()

    elif menu == "4":
        rate = calculate_rate(total_study_time, today_goal)
        reward_message(rate)

    elif menu == "5":
        print()
        print("공부 캐릭터 성장 프로그램을 종료합니다.")
        break

    else:
        print("잘못된 메뉴입니다. 다시 선택해주세요.")

```

...

3차 과제 조건 들어간 부분 정리

```
| 조건 | 코드에서 사용한 부분 |
|---|---|
| `while True` | 메뉴가 계속 반복되도록 사용 |
| `break` | 5번 종료 메뉴를 선택하면 프로그램 종료 |
| 함수 3개 이상 | `show_menu`, `calculate_exp`, `record_study`, `show_status`, `analyze_result`, `reward_message` 등 |
| 매개변수 | `calculate_exp(study_time, char_type)`, `calculate_rate(total, goal)` |
| `return` | 경험치, 총 공부 시간, 달성률 계산 결과를 돌려줌 |
| `global` | `record_study()` 안에서 레벨, 경험치, 능력치, 총 공부 시간을 수정할 때 사용 |
```

과제 설명 초안

이번 3차 과제에서는 2차 과제에서 만들었던 공부 캐릭터 성장 프로그램을 발전시켜 메뉴형 프로그램으로 구성하였다.

2차 과제에서는 사용자의 이름, 캐릭터 이름, 캐릭터 타입, 목표 공부 시간, 과목별 공부 시간 등을 입력받고 공부 결과를 출력하는 방식이었다.

3차 과제에서는 여기에 메뉴 기능을 추가하여 사용자가 원하는 기능을 직접 선택할 수 있도록 만들었다.

프로그램의 메뉴는 공부 기록 입력하기, 캐릭터 상태 보기, 공부 결과 분석하기, 보상 메시지 확인하기, 프로그램 종료하기로 구성하였다.

사용자가 공부 기록을 입력하면 공부한 과목과 시간이 리스트에 저장되고, 총 공부 시간과 경험치, 능력치가 증가한다. 경험치가 일정 수치 이상 올라가도록 하였다.

또한 목표 공부 시간과 실제 공부 시간을 비교하여 목표 달성률을 계산하고, 달성률에 따라 현실적인 보상 메시지를 출력하도록 만들었다.

보상은 게임 아이템이 아니라 실제로 자신에게 줄 수 있는 아이스크림, 음료, 간식 같은 보상으로 설정하여 공부 관리 프로그램의 현실성을 높였다.

2차 과제에서 발견한 점

2차 과제는 입력받은 공부 정보를 한 번 출력하는 방식에 가까웠다면, 3차 과제에서는 `while True`를 사용하여 프로그램이 계속 실행되도록 만들었다. 사용자는 메뉴를 선택하면서 여러 번 공부 기록을 추가할 수 있고, 현재 캐릭터 상태나 공부 결과를 다시 확인할 수 있다.

또한 코드를 기능별 함수로 나누어 구조를 더 깔끔하게 만들었다.

공부 기록 입력, 상태 보기, 결과 분석, 보상 메시지 출력 등을 각각 함수로 분리했기 때문에 코드가 더 읽기 쉬워졌다.

특히 `calculate\_exp`, `calculate\_total\_time`, `calculate\_rate` 함수에서는 매개변수를 사용하여 필요한 값을 전달하고, `return`을 사용하여 계산 결과를 다시 돌려받도록 하였다.

그리고 `record\_study()` 함수에서는 공부 시간, 경험치, 레벨, 능력치처럼 프로그램 전체에서 계속 바뀌는 값을 수정해야 하므로 `global`을 사용하였다.

보상 단계 이름

이번 프로그램에서는 공부 달성률에 따라 보상 단계를 4단계로 나누었다.

```
| 달성률 | 단계 이름 | 보상 |
|---|---|---|
| 40% 미만 | 출발 준비 단계 | 조금 더 공부하면 보상 가능 |
| 40% 이상 | 성장 시작 단계 | 작은 간식 |
| 70% 이상 | 집중 상승 단계 | 좋아하는 음료 |
| 100% 이상 | 목표 완주 단계 | 아이스크림 |
```

이렇게 구성하면 너무 유치하지 않으면서도, 공부 캐릭터가 성장하는 프로그램의 분위기와 잘 어울려.

사용자

```
조건에 안맞는 거 같아 user_name = ""
char_name = ""
char_type = "쉬엄쉬엄 공부" # 기본값
target_time = 1
today_goal = 1.0
```

```

level = 1
exp = 0
ability = 50

subjects = []
study_times = []

analysis_done = False

# 메뉴 출력 함수
def show_menu():
    print("\n" + "=" * 40)
    print("공부 캐릭터 성장 프로그램")
    print("=" * 40)
    print("1. 공부 정보 입력")
    print("2. 현재 정보 조회")
    print("3. 공부 결과 분석 및 보상 확인")
    print("4. 종료")

# 기본 정보 입력 함수
def input_basic_info():
    # [요구사항 5] 전역 변수 제어를 위해 global 키워드 사용
    global user_name, char_name, char_type, target_time, today_goal

    print("\n[기본 정보 입력]")
    user_name = input("사용자 이름을 입력하세요! : ")
    char_name = input("캐릭터 이름을 입력하세요! : ")

    # 공부 유형 선택 질문 서식 적용
    char_type = input("본인의 공부유형을 선택하시오(높은 집중력형/짧은 집중력형/쉬엄쉬엄 공부/스파르타형) : ")

    # 입력값 검증 및 예외 처리
    if not (char_type == "높은 집중력형" or char_type == "짧은 집중력형" or char_type == "쉬엄쉬엄 공부" or
char_type == "스파르타형"):
        print("잘못된 입력입니다. '쉬엄쉬엄 공부'로 설정합니다.")
        char_type = "쉬엄쉬엄 공부"

    target_time = int(input("전체 목표 시간을 입력하세요! : "))
    today_goal = float(input("하루 목표 시간을 입력하세요! : "))

    # 안전장치: 0 또는 음수 입력 시 에러 방지
    if target_time <= 0:
        print("잘못된 목표 시간입니다. 1시간으로 설정합니다.")
        target_time = 1

    if today_goal <= 0:
        print("잘못된 하루 목표 시간입니다. 1시간으로 설정합니다.")
        today_goal = 1.0

    print("기본 정보 입력이 완료되었습니다.")

# 공부 기록 입력 함수
def input_study_record():
    global analysis_done

    print("\n[공부 기록 입력]")
    subject = input("공부한 과목명을 입력하세요 : ")
    study_time = float(input(subject + " 공부 시간을 입력하세요 : "))

    subjects.append(subject)
    study_times.append(study_time)

    analysis_done = False
    print("공부 기록이 저장되었습니다.")

# 1번 메뉴 선택 시 실행될 통합 함수

```

```

def input_study_info():
    if user_name == "":
        input_basic_info()
    input_study_record()

# 현재 정보 조회 함수
def show_info():
    print("\n[현재 정보 조회]")

    if user_name == "":
        print("아직 입력된 기본 정보가 없습니다.")
    else:
        print("사용자 이름 :", user_name)
        print("캐릭터 이름 :", char_name)
        print("공부 유형 :", char_type)
        print("전체 목표 시간 :", target_time, "시간")
        print("하루 목표 시간 :", today_goal, "시간")
        print("레벨 :", level)
        print("경험치 :", exp)
        print("능력치 :", ability)

    print("\n[공부 기록]")

    if len(subjects) == 0:
        print("아직 입력된 공부 기록이 없습니다.")
    else:
        for i in range(len(subjects)):
            print(i + 1, "번째 기록 :", subjects[i], "/", study_times[i], "시간")

        sorted_times = sorted(study_times)
        print("공부 시간 정렬 :", sorted_times)

# [요구사항 3, 4] 매개변수와 return을 사용하는 함수
def calculate_total_time(times):
    total = 0 # 지역 변수
    for time in times:
        total += time
    return total

# [요구사항 3, 4] 매개변수와 return을 사용하는 함수
def calculate_rate(total, target):
    if target == 0:
        return 0.0
    rate = total / target * 100
    return rate

# 가장 많이 공부한 과목 찾기 함수
def find_best_subject(subject_list, time_list):
    best_index = time_list.index(max(time_list))
    best_subject = subject_list[best_index]
    return best_subject

# 보상 메시지 반환 함수
def get_reward_message(rate):
    if rate >= 100:
        return "공부 던전 클리어! 오늘의 보상으로 맛있는 한 끼를 선물하세요."
    elif rate >= 80:
        return "과제 격파대장! 오늘의 보상으로 디저트나 30분 휴식을 선물하세요."
    elif rate >= 50:
        return "집중력 예열러! 오늘의 보상으로 좋아하는 음료 한 잔을 선물하세요."
    elif rate > 0:
        return "책상 앞 착석 용사! 오늘의 보상으로 아이스크림 하나를 선물하세요."
    else:
        return "아직 보상이 없습니다. 먼저 공부 기록을 입력해보세요."

```

```

# 캐릭터 성장 처리 함수
def grow_character(rate):
    global level, exp, ability

    if rate >= 80:
        level += 1
        exp += 50
        ability += 10
        print("캐릭터 레벨업 성공!")
    elif rate >= 50:
        exp += 30
        ability += 5
        print("목표의 절반 이상 달성!")
    else:
        ability = max(0, ability - 5)
        print("목표 달성을 저조.. 공부량을 증가하세요!")

    if rate >= 80 and char_type == "스파르타형":
        exp += 20
        print("스파르타형 추가 보상 증정!")

# 공부 결과 분석 함수
def analyze_study():
    global analysis_done

    print("\n[공부 결과 분석 및 보상 확인]")

    if user_name == "":
        print("기본 정보를 먼저 입력해주세요.")
    elif len(study_times) == 0:
        print("공부 기록을 먼저 입력해주세요.")
    else:
        total_time = calculate_total_time(study_times)
        rate = calculate_rate(total_time, today_goal)
        best_subject = find_best_subject(subjects, study_times)
        reward = get_reward_message(rate)

        print("총 공부 시간 :", total_time, "시간")
        print("오늘의 목표 달성률 :", round(rate, 1), "%")
        print("가장 많이 공부한 과목 :", best_subject)

        if analysis_done == False:
            grow_character(rate)
            analysis_done = True
        else:
            print("이미 분석을 완료했습니다. 새 공부 기록을 입력하면 다시 성장할 수 있습니다.")

    print("\n[보상 메시지]")
    print(reward)

    print("\n[캐릭터 현재 상태]")
    print("레벨 :", level)
    print("경험치 :", exp)
    print("능력치 :", ability)

# =====
# [요구사항 1] 메인 로직 시작 안내 및 무한 루프
# =====

# 프로그램 실행 시 최초 1회 전체 기능 안내 문구 출력
print("=" * 50)
print("공부 캐릭터 성장 프로그램에 오신 것을 환영합니다! ")
print(" [안내] 본 프로그램은 다음과 같은 기능을 제공합니다.")
print(" 1. 공부 정보 입력 (유형 선택 및 당일 과목/시간 기록)")
print(" 2. 현재 정보 조회 (현재 스탯 및 공부 기록 정렬 조회)")
print(" 3. 공부 결과 분석 (목표 달성률 계산 및 보상/레벨업 확인)")
print(" 4. 프로그램 종료")
print("=" * 50)

```

```

while True:
    show_menu()

    choice = input("메뉴를 선택하세요 : ")

    # [요구사항 2] 함수 호출 위주의 간결한 메인 로직 구성
    if choice == "1":
        input_study_info()

    elif choice == "2":
        show_info()

    elif choice == "3":
        analyze_study()

    elif choice == "4":
        print("프로그램을 종료합니다. 오늘도 열공하느라 수고하셨습니다!")
        break # 종료 선택 시에만 break로 루프 탈출

    else:
        print("잘못된 입력입니다. 1~4 중에서 선택하세요.")

```

ChatGPT

코드 자체는 큰 틀에서 조건을 거의 만족하고 있어. 다만 교수님이 보기 좋게 하려면 "조건을 어디서 충족했는지"가 더 명확해야 하고, 몇 군데는 안정적이지 않아.

내가 보낸 코드는 사실 3차 과제 조건을 거의 다 만족해.
다만 내가 "조건에 안 맞는 것 같다"고 느낀 이유는 아마 조건이 코드 안에 있긴 한데, 교수님이 봤을 때 한눈에 드러나지 않는 부분이 있어서 그런 것 같아.

결론부터 말하면 이 코드는 조건에 맞아.

```

| 조건 | 네 코드에서 해당하는 부분 |
|---|---|
| `while True` | 맨 아래 메뉴 반복 부분 |
| `break` | 4번 종료 선택 시 사용 |
| 함수 3개 이상 | `show_menu`, `input_basic_info`, `show_info`, `calculate_rate` 등 많음 |
| 매개변수 | `calculate_total_time(times)`, `calculate_rate(total, target)` 등 |
| `return` | `calculate_total_time`, `calculate_rate`, `get_reward_message` 등 |
| `global` | `input_basic_info`, `grow_character`, `analyze_study` 등 |

```

그래서 조건 자체는 맞아.
다만 더 자연스럽게 다듬으면 좋아.

네 코드에서 수정하면 좋은 부분

1. `target\_time`이 거의 사용되지 않음

```

```python
target_time = int(input("전체 목표 시간을 입력하세요! : "))
```

```

이렇게 전체 목표 시간을 받는데, 분석에서는 주로 `today\_goal`만 사용하고 있어.

```

```python
rate = calculate_rate(total_time, today_goal)
```

```

즉, 전체 목표 시간은 입력받고 출력만 하고 있어서 조금 아쉬워.
그래서 분석할 때 전체 목표 달성률도 같이 출력하면 더 좋아.

2. 보상 단계가 조금 유치하게 느껴질 수 있음

현재 단계 이름은 이런 느낌이야.

```
```python
공부 던전 클리어러
과제 격파대장
집중력 예열러
책상 앞 착석 용사
```
```

조금 게임 느낌이 강해서, 네가 말한 것처럼 공부 성장 프로그램에 어울리게 바꾸면 더 좋아.

추천 단계는 이거야.

```
달성률	단계 이름	보상
100% 이상	목표 완주 단계	맛있는 한 끼 또는 아이스크림
80% 이상	집중 상승 단계	디저트 또는 30분 휴식
50% 이상	루틴 형성 단계	좋아하는 음료
0% 초과	시작 점검 단계	작은 간식 또는 아이스크림
```

3. `global`은 잘 쓰고 있음

네 프로그램에서 전역변수로 둔 값들은 괜찮아.

```
```python
user_name
char_name
char_type
target_time
today_goal
level
exp
ability
subjects
study_times
analysis_done
```
```

특히 `global`은 이런 함수에서 쓰는 게 맞아.

```
```python
def input_basic_info():
 global user_name, char_name, char_type, target_time, today_goal
```
```

기본 정보를 함수 안에서 바꾸니까 필요해.

```
```python
def grow_character(rate):
 global level, exp, ability
```
```

레벨, 경험치, 능력치를 함수 안에서 바꾸니까 필요해.

```
```python
def analyze_study():
 global analysis_done
```
```

분석 완료 여부를 함수 안에서 바꾸니까 필요해.

다만 `subjects.append()`나 `study\_times.append()`처럼 리스트 안에 값을 추가하는 건 꼭 `global`이 없어도 돼. 리스트 자체를 새로 바꾸는 게 아니라, 기존 리스트 안에 값을 추가하는 거라서 괜찮아.

수정한 최종 코드 초안

아래 코드는 네가 보낸 코드를 최대한 유지하면서, 조건이 더 잘 보이게 다듬은 버전이야.

```

```python
user_name = ""
char_name = ""
char_type = "쉬엄쉬엄 공부"
target_time = 1
today_goal = 1.0

level = 1
exp = 0
ability = 50

subjects = []
study_times = []

analysis_done = False

메뉴 출력 함수
def show_menu():
 print("\n" + "=" * 40)
 print("공부 캐릭터 성장 프로그램")
 print("=" * 40)
 print("1. 공부 정보 입력")
 print("2. 현재 정보 조회")
 print("3. 공부 결과 분석 및 보상 확인")
 print("4. 종료")

기본 정보 입력 함수
def input_basic_info():
 global user_name, char_name, char_type, target_time, today_goal

 print("\n[기본 정보 입력]")
 user_name = input("사용자 이름을 입력하세요! : ")
 char_name = input("캐릭터 이름을 입력하세요! : ")

 char_type = input("본인의 공부유형을 선택하시오(높은 집중력형/짧은 집중력형/쉬엄쉬엄 공부/스파르타형) : ")

 if not (char_type == "높은 집중력형" or char_type == "짧은 집중력형" or char_type == "쉬엄쉬엄 공부" or
char_type == "스파르타형"):
 print("잘못된 입력입니다. '쉬엄쉬엄 공부'로 설정합니다.")
 char_type = "쉬엄쉬엄 공부"

 target_time = int(input("전체 목표 시간을 입력하세요! : "))
 today_goal = float(input("하루 목표 시간을 입력하세요! : "))

 if target_time <= 0:
 print("잘못된 전체 목표 시간입니다. 1시간으로 설정합니다.")
 target_time = 1

 if today_goal <= 0:
 print("잘못된 하루 목표 시간입니다. 1시간으로 설정합니다.")
 today_goal = 1.0

 print("기본 정보 입력이 완료되었습니다.")

공부 기록 입력 함수
def input_study_record():
 global analysis_done

 print("\n[공부 기록 입력]")
 subject = input("공부한 과목명을 입력하세요 : ")
 study_time = float(input(subject + " 공부 시간을 입력하세요 : "))

 if study_time <= 0:
 print("공부 시간은 0보다 커야 합니다.")
 else:
 subjects.append(subject)
 study_times.append(study_time)

```



```

 analysis_done = False
 print("공부 기록이 저장되었습니다.")

1번 메뉴 선택 시 실행될 통합 함수
def input_study_info():
 if user_name == "":
 input_basic_info()

 input_study_record()

현재 정보 조회 함수
def show_info():
 print("\n[현재 정보 조회]")

 if user_name == "":
 print("아직 입력된 기본 정보가 없습니다.")
 else:
 print("사용자 이름 :", user_name)
 print("캐릭터 이름 :", char_name)
 print("공부 유형 :", char_type)
 print("전체 목표 시간 :", target_time, "시간")
 print("하루 목표 시간 :", today_goal, "시간")
 print("레벨 :", level)
 print("경험치 :", exp)
 print("능력치 :", ability)

 print("\n[공부 기록]")

 if len(subjects) == 0:
 print("아직 입력된 공부 기록이 없습니다.")
 else:
 for i in range(len(subjects)):
 print(i + 1, "번째 기록 :", subjects[i], "/", study_times[i], "시간")

 sorted_times = sorted(study_times)
 print("공부 시간 정렬 :", sorted_times)

매개변수와 return을 사용하는 함수 1
def calculate_total_time(times):
 total = 0
 for time in times:
 total += time
 return total

매개변수와 return을 사용하는 함수 2
def calculate_rate(total, target):
 if target == 0:
 return 0.0

 rate = total / target * 100
 return rate

매개변수와 return을 사용하는 함수 3
def find_best_subject(subject_list, time_list):
 best_index = time_list.index(max(time_list))
 best_subject = subject_list[best_index]
 return best_subject

매개변수와 return을 사용하는 함수 4
def get_reward_message(rate):
 if rate >= 100:
 return "목표 완주 단계! 오늘의 보상으로 아이스크림이나 맛있는 한 끼를 선물하세요."
 elif rate >= 80:
 return "집중 상승 단계! 오늘의 보상으로 디저트나 30분 휴식을 선물하세요."

```

```

elif rate >= 50:
 return "루틴 형성 단계! 오늘의 보상으로 좋아하는 음료 한 잔을 선물하세요."
elif rate > 0:
 return "시작 점검 단계! 오늘의 보상으로 작은 간식 하나를 선물하세요."
else:
 return "아직 보상이 없습니다. 먼저 공부 기록을 입력해보세요."

캐릭터 성장 처리 함수
def grow_character(rate):
 global level, exp, ability

 if rate >= 100:
 level += 2
 exp += 70
 ability += 15
 print("목표를 완주하여 캐릭터가 크게 성장했습니다!")

 elif rate >= 80:
 level += 1
 exp += 50
 ability += 10
 print("캐릭터 레벨업 성공!")

 elif rate >= 50:
 exp += 30
 ability += 5
 print("목표의 절반 이상을 달성했습니다!")

 else:
 ability = max(0, ability - 5)
 print("목표 달성률이 낮습니다. 공부량을 조금 더 늘려보세요!")

 if rate >= 80 and char_type == "스파르타형":
 exp += 20
 print("스파르타형 추가 경험치가 지급되었습니다!")

공부 결과 분석 함수
def analyze_study():
 global analysis_done

 print("\n[공부 결과 분석 및 보상 확인]")

 if user_name == "":
 print("기본 정보를 먼저 입력해주세요.")

 elif len(study_times) == 0:
 print("공부 기록을 먼저 입력해주세요.")

 else:
 total_time = calculate_total_time(study_times)

 today_rate = calculate_rate(total_time, today_goal)
 total_rate = calculate_rate(total_time, target_time)

 best_subject = find_best_subject(subjects, study_times)
 reward = get_reward_message(today_rate)

 print("총 공부 시간 :", total_time, "시간")
 print("하루 목표 달성률 :", round(today_rate, 1), "%")
 print("전체 목표 달성률 :", round(total_rate, 1), "%")
 print("가장 많이 공부한 과목 :", best_subject)

 if analysis_done == False:
 grow_character(today_rate)
 analysis_done = True
 else:
 print("이미 분석을 완료했습니다. 새 공부 기록을 입력하면 다시 성장할 수 있습니다.")

```

```

 print("\n[보상 메시지]")
 print(reward)

 print("\n[캐릭터 현재 상태]")
 print("레벨 :", level)
 print("경험치 :", exp)
 print("능력치 :", ability)

print("=" * 50)
print("공부 캐릭터 성장 프로그램에 오신 것을 환영합니다!")
print("[안내] 본 프로그램은 다음과 같은 기능을 제공합니다.")
print("1. 공부 정보 입력")
print("2. 현재 정보 조회")
print("3. 공부 결과 분석 및 보상 확인")
print("4. 프로그램 종료")
print("=" * 50)

while True를 사용한 메뉴 반복
while True:
 show_menu()

 choice = input("메뉴를 선택하세요 : ")

 if choice == "1":
 input_study_info()

 elif choice == "2":
 show_info()

 elif choice == "3":
 analyze_study()

 elif choice == "4":
 print("프로그램을 종료합니다. 오늘도 열공하느라 수고하셨습니다!")
 break

 else:
 print("잘못된 입력입니다. 1~4 중에서 선택하세요.")
...

```

# 이 코드가 조건에 맞는 이유

과제 설명에는 이렇게 적으면 돼.

```text

이번 3차 과제에서는 2차 과제에서 만들었던 공부 캐릭터 성장 프로그램을 메뉴형 프로그램으로 발전시켰다.

while True를 사용하여 사용자가 종료를 선택하기 전까지 메뉴가 반복되도록 만들었고, 사용자가 4번 종료 메뉴를 선택하면 break를 사용하여 반복 종료를 하도록 하였다.

또한 show_menu, input_basic_info, input_study_record, show_info, analyze_study 등 여러 개의 함수를 만들어 기능별로 코드를 나누었다.

calculate_total_time, calculate_rate, find_best_subject, get_reward_message 함수에서는 매개변수를 사용하여 필요한 값을 전달하고, return을 사용하여 계산 결과나 메시지를 반환하도록 하였다.

프로그램 전체에서 계속 유지되어야 하는 사용자 이름, 캐릭터 정보, 목표 시간, 레벨, 경험치, 능력치 등은 전역변수로 두었다.

특히 input_basic_info 함수에서는 기본 정보를 수정해야 하므로 global을 사용했고, grow_character 함수에서는 level, exp, ability 값을 변경해야 하므로 global을 사용하였다.

```

---

정리하면 네 코드가 조건에 안 맞는 게 아니라, 조금만 다듬으면 조건이 더 확실하게 보이는 코드가 돼.

특히 지금 수정한 버전은 `while True`, `break`, 함수, 매개변수, `return`, `global`이 전부 명확하게 들어가 있어서 3차 과제용으로 괜찮아.

## 사용자

이 내용들을 피디에프로 내보내려면 어떻게 해야해

## ChatGPT

PDF로 제출하려면 보통 "코드 파일"과 "과제 설명 내용"을 문서에 붙여 넣고 저장하면 돼. 제출 형식에 따라 가장 쉬운 방법을 기준으로 설명해줄

가능해. 방법은 크게 2가지가 있어.

---

# 1. 제일 쉬운 방법: 채팅 내용을 바로 PDF로 저장

지금 대화 내용을 그대로 PDF로 저장하고 싶으면:

1. 이 대화에서 필요한 부분을 드래그해서 복사
2. 한글, Word, 또는 Google Docs에 붙여넣기
3. 제목 정리
4. `파일` → `내보내기/다운로드` → `PDF` 선택

또는 브라우저에서 바로 할 수도 있어.

1. 키보드에서 `Ctrl + P` 누르기
2. 프린터 선택에서 PDF로 저장 선택
3. 저장

근데 이 방법은 채팅 UI까지 같이 들어가거나 정리가 덜 예뻐 수 있어서, 과제 제출용이면 아래 방법이 더 좋아.

---

# 2. 과제 제출용 추천 방법: Word나 한글에 정리해서 PDF 저장

과제용이면 문서를 이렇게 구성하면 좋아.

```text

제목: 공부 캐릭터 성장 프로그램 3차 과제

1. 프로그램 주제
 2. 2차 과제에서 발전한 점
 3. 3차 과제 조건 적용 내용
 4. 주요 기능 설명
 5. 전체 코드
 6. 실행 예시 또는 느낀 점
- ```

문서에 넣을 내용 예시

1. 프로그램 주제

```text

이번 프로그램은 사용자의 공부 기록을 입력받고, 공부 시간에 따라 캐릭터가 성장하는 공부 캐릭터 성장 프로그램이다. 사용자는 메뉴를 통해 공  
현재 캐릭터 상태를 조회하며, 공부 결과 분석과 보상 메시지를 확인할 수 있다.

```

2. 2차 과제에서 발전한 점

```text

2차 과제에서는 사용자 이름, 캐릭터 이름, 공부 유형, 목표 시간, 과목별 공부 시간을 입력받고 결과를 출력하는 프로그램을 만들었다. 3차 과제  
발전시켜 메뉴형 프로그램으로 구성하였다. 사용자가 원하는 메뉴를 선택하여 공부 기록을 추가하거나 현재 정보를 조회할 수 있도록 하였고, 공  
메시지 기능도 추가하였다.

```

3. 3차 과제 조건 적용 내용

```

| 조건 | 적용 내용 |
|---|---|
| `while True` | 사용자가 종료를 선택하기 전까지 메뉴가 계속 반복되도록 사용 |
| `break` | 4번 종료 메뉴를 선택하면 반복문을 종료할 때 사용 |
| 함수 3개 이상 | 메뉴 출력, 정보 입력, 공부 기록, 결과 분석 등 여러 기능을 함수로 분리 |
| 매개변수 | `calculate_rate(total, target)`처럼 함수에 값을 전달할 때 사용 |
| `return` | 계산한 총 공부 시간, 달성률, 보상 메시지를 다시 돌려줄 때 사용 |
| `global` | 함수 안에서 전역변수 값을 수정할 때 사용 |

```

4. 주요 기능 설명

```
```text
```

이 프로그램은 크게 네 가지 메뉴로 구성되어 있다.

1번 메뉴는 공부 정보 입력 기능이다. 처음 실행할 때는 사용자 이름, 캐릭터 이름, 공부 유형, 목표 시간을 입력받고 이후에는 공부한 과목과 시간

2번 메뉴는 현재 정보 조회 기능이다. 사용자의 기본 정보, 캐릭터 레벨, 경험치, 능력치, 공부 기록을 확인할 수 있다.

3번 메뉴는 공부 결과 분석 및 보상 확인 기능이다. 총 공부 시간과 목표 달성률을 계산하고, 가장 많이 공부한 과목을 찾아 출력한다. 또한 달성률 메시지를 제공하고 캐릭터를 성장시킨다.

4번 메뉴는 프로그램 종료 기능이다. 사용자가 종료를 선택하면 break를 사용하여 while True 반복문을 빠져나간다.

```
```
```

5. 전체 코드

여기에 네 최종 코드를 붙여 넣으면 돼.

코드는 문서에서 글꼴을 Consolas 또는 Courier New로 바꾸면 코드처럼 보여서 깔끔해.

PDF 저장 방법

Word 기준

1. Word에 내용 붙여넣기
2. 제목, 표, 코드 정리
3. `파일` 클릭
4. `다른 이름으로 저장`
5. 파일 형식을 PDF로 선택
6. 저장

한글 기준

1. 한글에 내용 붙여넣기
2. 코드 부분은 글꼴을 `Consolas` 비슷한 고정폭 글꼴로 설정
3. `파일` 클릭
4. `PDF로 저장하기` 또는 `다른 이름으로 저장`
5. PDF 선택 후 저장

Google Docs 기준

1. Google Docs 새 문서 만들기
2. 내용 붙여넣기
3. `파일` 클릭
4. `다운로드` 클릭
5. `PDF 문서(.pdf)` 선택

제출용으로 제일 추천하는 방식

과제 제출용이면 그냥 채팅을 PDF로 저장하지 말고,
내가 정리해준 내용을 Word나 한글에 옮겨서 이렇게 제출하는 게 좋아.

```
```text
1. 과제 설명
2. 조건 적용 내용
3. 코드
4. 실행 결과 설명
5. 느낀 점
```
```

파일 이름은 교수님이 따로 정한 형식이 없다면 보통 이렇게 하면 돼.

```
```text
이름_학과_학번_3차과제.pdf
```
```

코드는 따로 `.py` 파일로도 저장해두는 게 좋아. PDF에는 설명과 코드가 들어가고, 실제 실행 파일은 `3차과제.py`처럼 따로 보관하면 돼.

사용자

아니 너랑 나눈 이 대화 전체를 피디에프로